Università di Parma
Corso di High Performance Computing

Sintesi di circuiti quantistici con PPO, Transformer e HPC

Relazione sul progetto

Alessandro La Mantia

Matricola: 399269

Introduzione

In questo progetto ho studiato come costruire un circuito quantistico che prepari uno stato desiderato usando pochi gate. Il circuito parte sempre da $|00\ldots0\rangle$. Ho usato un agente di reinforcement learning per scegliere i gate e un ottimizzatore per regolare gli angoli delle rotazioni.

Ho lavorato su Bell, GHZ e due stati target random, a due e tre qubit. Per ogni caso ho confrontato i risultati con circuiti casuali. Ho usato il cluster HPC per eseguire gli addestramenti. In questa relazione spiego la struttura del progetto, il funzionamento del codice e i risultati che ho riportato nella presentazione.

Indice

1. Obiettivo e strumenti
2. Struttura delle cartelle
3. Caricamento dei target
4. Ambiente e azioni
5. Calcolo della reward
6. Ottimizzazione degli angoli
7. Transformer e PPO
8. Training e valutazione
9. Esecuzione sul cluster HPC
10. Risultati
11. Conclusioni e possibili miglioramenti

1. Obiettivo e strumenti

Lo stato target è lo stato quantistico che voglio ottenere. Per misurare quanto il circuito si avvicina al target uso la fidelity F, che varia tra 0 e 1. Un valore vicino a 1 indica un buon risultato. Nel codice uso anche l'infedeltà d = 1 − F: più è bassa, meglio è.

```
F = |⟨ψ_target | ψ_circuito⟩|²
d = 1 − F
Successo nell'ambiente: d < 0,01
```

Nei report considero riusciti i tentativi con F ≥ 0,99. L'ambiente termina quando d < 0,01, quindi con F > 0,99. L'obiettivo è raggiungere questa precisione con meno gate possibili. La reward premia entrambi gli aspetti, ma non garantisce di trovare sempre il circuito più corto possibile.

Strumento	**Uso nel progetto**
Gymnasium	Definisco ambiente, azioni e osservazioni.
PennyLane	Simulo i circuiti e calcolo i gradienti degli angoli.
PyTorch	Implemento la rete e ottimizzo gli angoli con Adam.
Ray RLlib	Uso PPO e raccolgo esperienze con più worker.
Qiskit	Genero la baseline di circuiti casuali.
SLURM	Invio gli esperimenti al cluster HPC.

Ho usato simulazioni classiche di stati puri, senza rumore. Non ho eseguito i circuiti su un computer quantistico reale. Circuiti brevi restano comunque interessanti perché, su un dispositivo reale, ogni gate può introdurre errori.

2. Struttura delle cartelle

Ho diviso il progetto in cinque script principali e tre moduli nella cartella quantum_rl. Gli script gestiscono gli esperimenti; i moduli contengono target, ambiente e rete.

```
Definitivo/
├── 1.pre_trained_table.py
├── 2.train.py
├── 3.run_repeated_training.py
├── 4.plot_training_summary.py
├── 5.evaluate.py
├── quantum_rl/
│   ├── __init__.py
│   ├── 1.targets.py
│   ├── 2.environment.py
│   └── 3.model.py
├── targets/
│   ├── bell.json
│   ├── ghz.json
│   ├── random_target_2_qubit_normalized.json
│   ├── random_target_3_qubit_normalized.json
│   ├── random_target_not_normalized.json
│   └── invalid_wrong_dimension_target.json
├── plots/
│   ├── bell/
│   ├── ghz/
│   ├── random2/
│   ├── random3/
│   └── matplotlib_cache/
├── checkpoints/
├── environment-hpc.yml
├── requirements.txt
├── job_train.slurm
├── README.md
├── .vscode/
├── __pycache__/
├── pdf_browser_audit/
└── relazione/
```

File	A cosa serve
1.pre_trained_table.py	Costruisce circuiti casuali e salva fidelity, gate, esito e tempo in un CSV.
2.train.py	Avvia PPO, registra le metriche e salva i checkpoint.
3.run_repeated_training.py	Ripete training e valutazione più volte e crea una tabella finale.
4.plot_training_summary.py	Legge i CSV e crea i grafici di reward e fidelity.
5.evaluate.py	Carica una policy addestrata e misura i risultati su più episodi.
quantum_rl/1.targets.py	Carica, controlla e normalizza lo stato target.
quantum_rl/2.environment.py	Gestisce circuito, azioni, angoli, simulazione e reward.
quantum_rl/3.model.py	Definisce il Transformer che propone le azioni.

File	A cosa serve
quantum_rl/__init__.py	Permette di importare i moduli numerati con nomi semplici.

In targets salvo gli stati da preparare. In plots raccolgo i grafici, separati per esperimento. I checkpoint servono a salvare e ricaricare l'agente. requirements.txt elenca le librerie; environment-hpc.yml definisce l'ambiente Conda; job_train.slurm contiene il comando da eseguire sul cluster.

Le cartelle .vscode, __pycache__, matplotlib_cache e pdf_browser_audit contengono impostazioni o file di supporto. Non fanno parte dell'algoritmo. In relazione salvo questo documento e i file usati per generarlo.

I nomi dei moduli iniziano con un numero. Per importarli, __init__.py usa importlib e li espone come quantum_rl.targets, quantum_rl.environment e quantum_rl.model. In questo modo mantengo un solo file sorgente per ciascun modulo.

3. Caricamento dei target

Ho previsto tre tipi di target. Bell per gli esperimenti a 2 qubit, GHZ per 3 qubit, e, infine, stati random a 2 e 3 qubit sono stati testati.

```
Bell = (|00⟩ + |11⟩) / √2
GHZ₃ = (|000⟩ + |111⟩) / √2
```

La funzione `load_target_state()` (quantum_rl/1.targets.py) legge file JSON o NumPy (.npy). Nel JSON ogni ampiezza può essere un numero reale oppure una coppia [parte reale, parte immaginaria]. Questo esempio rappresenta Bell:

```
[[0.7071067811865475, 0], [0, 0],
 [0, 0], [0.7071067811865475, 0]]
```

`validate_target_state()` (quantum_rl/1.targets.py) converte i dati in un vettore complesso e ne controlla la dimensione: per n qubit servono 2^n ampiezze. Successivamente, il vettore viene normalizzato, quindi si può anche fornire un vettore non normalizzato.

quantum_rl/1.targets.py — estratto di validate_target_state(); np.asarray e np.linalg.norm sono funzioni NumPy.

```
vector = np.asarray(state, dtype=np.complex128).reshape(-1)
norm = np.linalg.norm(vector)
# Dopo i controlli su dimensione e norma:
return vector / norm, inferred_qubits
```

Se specifico sia il file sia il nome di un target, il programma usa il file. Il numero di qubit viene dedotto dalla dimensione e confrontato con quello eventualmente indicato da riga di comando.

4. Ambiente e azioni

4.1 Come rappresento il circuito

La classe QuantumCircuitEnv, in quantum_rl/2.environment.py, contiene il circuito corrente. In current_gates salvo la sequenza dei gate; in rotation_angles salvo gli angoli di RX, RY e RZ. Ogni gate è descritto da GateAction, definita nello stesso file, con nome e qubit coinvolti.

Gate	Scelta dell'agente	Angolo
RX, RY, RZ	Tipo di rotazione e qubit	Lo ottimizzo con Adam.
H	Qubit su cui applicare Hadamard	Non ha un parametro da ottimizzare.
CNOT	Qubit di controllo e qubit target	Non ha un parametro da ottimizzare.

Ogni azione è un numero intero che identifica un gate del catalogo. Per n qubit ci sono 3n rotazioni, n Hadamard e n(n−1) CNOT diretti: dieci azioni per due qubit e diciotto per tre. L'agente sceglie il gate, mentre gli angoli vengono calcolati separatamente.

4.2 Un episodio, dall'inizio alla fine

Per spiegare il ciclo considero un esempio con la preparazione dello stato di Bell. Dato il target fisso, (|00⟩ + |11⟩)/√2, lo stato ottenuto ad ogni step dall'agente, dopo avere aggiunto un gate, viene confrontato con lo stato target, finché non raggiunge la fidelity desiderata.

Primo passaggio: preparo il punto di partenza

Chiamo `reset()` (quantum_rl/2.environment.py). Cancello i gate del tentativo precedente, svuoto gli angoli e simulo il circuito vuoto. Ottengo lo stato |00⟩. Chiamo poi `_distance()` (quantum_rl/2.environment.py) per misurare l'errore iniziale: per Bell la fidelity è 0,5 e l'infedeltà è 0,5. Salvo questo errore in prev_dist, perché mi servirà dopo il primo gate.

quantum_rl/2.environment.py — estratto di reset(); le funzioni self.* richiamate sono nello stesso file, np.empty appartiene a NumPy.

```
self.current_gates = []
self.rotation_angles = np.empty(0, dtype=np.float64)
self.reached_milestones.clear()
self.current_state = self._simulate()
self.prev_dist = self._distance()
return self._get_obs(), self._get_info()
```

Con `_get_obs()` (quantum_rl/2.environment.py) preparo l'osservazione iniziale da dare alla rete. Con `_get_info()` (quantum_rl/2.environment.py) preparo invece le informazioni leggibili, come fidelity e lista dei gate. L'osservazione serve a decidere; info serve a controllare il risultato.

Secondo passaggio: l'agente sceglie un gate

Ora devo decidere quale gate aggiungere. L'osservazione è una scheda del tentativo: descrive dove sono arrivato, quali gate ho già usato e quanto sono lontano dal target. La rete legge questa scheda con `forward()` (quantum_rl/3.model.py) e assegna un punteggio alle mosse possibili. All'inizio non conosce già la soluzione: impara dopo molti tentativi.

RLlib trasforma i punteggi in probabilità di scelta. Per esempio, potrebbe dare a H(0) una probabilità del 60% e dividere il resto fra altre mosse ammesse. È un esempio inventato, non un valore misurato. Durante il training posso provare anche mosse meno probabili; senza esplorazione scelgo quella più probabile.

Suppongo venga scelto H(0), cioè Hadamard sul qubit zero. È il primo dei due qubit: la numerazione parte da zero. H crea una sovrapposizione, quindi il circuito non resta soltanto in |00⟩. Non impongo questa scelta all'agente: la uso per seguire un episodio concreto.

La scelta dell'azione viene richiesta a RLlib: in 5.evaluate.py uso il suo metodo compute_single_action(). Questo metodo appartiene alla libreria, mentre il calcolo dei punteggi della mia rete è in `forward()` (quantum_rl/3.model.py).

Terzo passaggio: aggiungo il gate al circuito

La scelta arriva come numero, non come testo "H(0)". Uso un catalogo, simile a un elenco numerato: con due qubit l'indice 6 corrisponde a H(0). Passo quindi 6 a `step(action)` (quantum_rl/2.environment.py). La funzione controlla che la mossa sia ammessa, legge la voce 6 e la aggiunge a current_gates. Questa lista è la ricetta del circuito: per ora contiene soltanto H(0).

Se scegliessi RX, RY o RZ, dovrei decidere anche quanto ruotare il qubit. L'agente sceglie il tipo di rotazione; il programma aggiunge un angolo zero e lo regola con Adam. Per H e CNOT non devo cercare un angolo.

quantum_rl/2.environment.py — estratto di step(); append aggiunge alla lista e np.append appartiene a NumPy.

```
gate = self.actions[action]
self.current_gates.append(gate)
if gate.name in ROTATIONS:
    self.rotation_angles = np.append(self.rotation_angles, 0.0)
```

Se ci sono angoli, `step(action)` (quantum_rl/2.environment.py) richiama `_optimise_angles()` (quantum_rl/2.environment.py) e confronta i tentativi descritti nella sezione 6. In questo esempio ho soltanto H, quindi salto l'ottimizzazione e passo alla simulazione.

Quarto passaggio: calcolo cosa ha prodotto il circuito

Con `_simulate()` (quantum_rl/2.environment.py) eseguo virtualmente la ricetta e vedo il risultato. Dopo H(0) ho (|00⟩ + |10⟩)/√2. Se misurassi i qubit, potrei leggere 00 oppure 10, ciascuno con probabilità del 50%. Bell richiede invece 00 oppure 11, con le corrette ampiezze: manca ancora il collegamento tra i qubit.

Con `_distance()` (quantum_rl/2.environment.py) confronto lo stato completo con Bell, tenendo conto anche delle ampiezze complesse e non soltanto delle probabilità di misura. Ottengo F=0,25, quindi d=1−0,25=0,75. Sono momentaneamente più lontano dal target, anche se H servirà per completare il circuito.

A questo punto `step(action)` (quantum_rl/2.environment.py) confronta il nuovo errore con quello precedente e calcola la reward. In questa prima mossa l'errore aumenta, quindi il premio è negativo. Preparo poi la nuova osservazione con `_get_obs()` (quantum_rl/2.environment.py) e la restituisco all'agente.

Quinto passaggio: uso il risultato per la scelta successiva

L'agente legge la scheda aggiornata e suppongo scelga CNOT(0,1). Il qubit zero controlla il qubit uno: quando il controllo vale 1, il target viene invertito. Quindi |00⟩ resta com'è e |10⟩ diventa |11⟩. Ottengo (|00⟩ + |11⟩)/√2: è Bell.

Passo a `step(action)` (quantum_rl/2.environment.py) l'indice 8, che a due qubit identifica questo CNOT. La ricetta contiene ora H(0) e CNOT(0,1). La risimulo e ottengo fidelity idealmente 1. Assegno i bonus di successo e chiudo il tentativo: non serve aggiungere altri gate.

quantum_rl/2.environment.py — passaggi di step(); _simulate() e _distance() sono definiti nello stesso script.

```
self.current_state = self._simulate()
distance = self._distance()
# Qui calcolo la reward, descritta nella sezione 5.
terminated = distance < self.threshold
truncated = len(self.current_gates) >= self.max_gates
```

terminated segnala il successo; truncated segnala che ho esaurito il budget. Se non raggiungo nessuna delle due condizioni, l'agente riceve l'osservazione aggiornata e sceglie un altro gate. PPO raccoglie queste mosse e le reward e, dopo aver raccolto un gruppo di esperienze, aggiorna i pesi della rete.

4.3 Simulazione e osservazione

Il simulatore è il banco di prova: gli consegno la ricetta e lui calcola lo stato prodotto. `_simulate()` (quantum_rl/2.environment.py) passa gli angoli a `_state_torch()` (quantum_rl/2.environment.py), che legge current_gates. Se trova H e CNOT, parte da |00⟩ e applica prima H, poi CNOT. A ogni prova riparte dall'inizio, senza riapplicare H allo stato finale precedente. Uso il simulatore PennyLane default.qubit.

quantum_rl/2.environment.py — estratto del circuito interno a _state_torch(); le operazioni qml.* appartengono a PennyLane.

```
for gate in gates:
    if gate.name == "CNOT":
        qml.CNOT(wires=gate.wires)
    elif gate.name == "H":
        qml.Hadamard(wires=gate.wires[0])
    else:
        getattr(qml, gate.name)(parameters[parameter_index],
                                wires=gate.wires[0])
        parameter_index += 1
return qml.state()
```

Il ciclo legge un'istruzione della ricetta alla volta. Per H e CNOT prende i qubit indicati; per una rotazione prende anche l'angolo. Alla fine restituisce le ampiezze, cioè i numeri che descrivono lo stato. Durante la ricerca degli angoli tengo anche le informazioni necessarie per capire come cambiarli. Per una semplice verifica disattivo le informazioni relative al gradiente con torch.no_grad(), una funzione PyTorch usata in _simulate() (quantum_rl/2.environment.py) in quantum_rl/2.environment.py.

Con `_get_obs()` (quantum_rl/2.environment.py) preparo la scheda da dare alla rete. Deve contenere numeri, perché la rete non legge la ricetta a parole. Inserisco le ampiezze, separando parte reale e immaginaria, la quota di gate già usati, l'errore e la storia delle scelte con gli angoli. Le caselle della storia ancora vuote valgono zero.

```
osservazione = [Re(ψ), Im(ψ), gate_usati / max_gates, d, storia]
Dimensione = 2·2ⁿ + 2 + 2·max_gates
```

Ho incluso la storia perché due circuiti possono produrre lo stesso stato, ma avere parametri diversi da riottimizzare. L'osservazione contiene anche action_mask, che indica quali azioni posso scegliere. Uso lo statevector completo perché lavoro in simulazione.

Per esempio, dopo il primo H l'osservazione indica che ho già usato un gate, che l'errore vale 0,75 e che nella storia compare H(0). Il target resta sempre lo stesso; cambiano il circuito costruito e i dati che descrivono il tentativo.

4.4 Come blocco le azioni inutili

`_action_mask()` (quantum_rl/2.environment.py) prepara un elenco di permessi: 1 significa "puoi scegliere questa mossa", 0 significa "è bloccata". All'inizio un CNOT non cambia |00⟩ perché il controllo è zero. Lo blocco finché non inserisco gate che possono attivare il controllo. Blocco anche RZ su qubit inattivi: applicato a |0⟩, da solo non crea la sovrapposizione che mi serve.

Valuto l'attività guardando la ricetta. RX, RY e H rendono un qubit potenzialmente attivo, anche se un angolo attuale vale zero: potrebbe essere cambiato più avanti. Un CNOT con controllo attivo può coinvolgere anche il suo target.

Blocco anche gate identici ripetuti, compresi quelli separati solo da operazioni su qubit diversi. Due H identici si annullano e due rotazioni consecutive sullo stesso asse possono essere unite. La maschera aiuta quindi l'agente a evitare sequenze ridondanti. `step()` (quantum_rl/2.environment.py) verifica sempre che l'azione scelta sia ammessa.

All’inizio, per esempio, CNOT(0,1) è bloccato perché il controllo è ancora fermo in |0⟩. Dopo H(0) il controllo è attivo e posso scegliere quel CNOT. La maschera evita quindi una prima mossa senza effetto, lasciando disponibile il gate quando può servire.

5. Calcolo della reward

Dopo ogni gate devo dire all'agente se la scelta è stata utile. Per questo calcolo la reward dentro `step()` (quantum_rl/2.environment.py). È un punteggio: non modifica lo stato quantistico e non ottimizza gli angoli. PPO la usa per imparare quali scelte conviene ripetere.

Per calcolarla mi servono l'errore prima del gate, salvato in prev_dist, e quello dopo il gate e l'eventuale ottimizzazione, salvato in distance. A questi sottraggo il costo del gate per preferire soluzioni più corte e, infine, vengono aggiunti gli eventuali premi. Tutti i valori seguenti usano i parametri standard del codice.

```
reward = 10 · ln((d_precedente + ε) / (d_attuale + ε)) − 1
         + bonus_soglia + bonus_finale
ε = 10⁻¹²
```

5.1 Miglioramento e costo del gate

Il primo termine è positivo se l'errore diminuisce e negativo se aumenta. Il logaritmo premia il miglioramento relativo: dimezzare l'errore dà circa 6,93 punti. Ogni gate costa un punto, anche se non cambia lo stato. ε evita di calcolare il logaritmo di zero.

Se passo da fidelity 0,80 a 0,90, l'infedeltà scende da 0,20 a 0,10: ho dimezzato l'errore e ricevo circa 5,93 punti dopo il costo del gate. Lo stesso premio vale passando da errore 0,10 a 0,05. Non premio quindi soltanto la differenza numerica di fidelity, ma quanto ho ridotto l'errore rispetto a prima.

Cambiamento dell'errore	Reward prima degli altri bonus
Da 0,2 a 0,1	$10 \ln(2) - 1 \approx +5{,}93$
Errore invariato	-1
Da 0,1 a 0,2	$10 \ln(0{,}5) - 1 \approx -7{,}93$

quantum_rl/2.environment.py — estratto di step(); math.log è il logaritmo della libreria standard Python.

```
epsilon = 1e-12
reward = (
    -math.log(distance + epsilon)
    + math.log(self.prev_dist + epsilon)
) * self.loss_mult - self.step_penalty
self.prev_dist = distance
```

Nel codice loss_mult vale 10 e step_penalty vale 1. Aggiorno prev_dist soltanto dopo il confronto: così, al prossimo gate, il punto di partenza sarà l'errore appena ottenuto. Se lo aggiornassi prima, confronterei lo stesso valore con sé stesso e perderei il segnale di miglioramento.

5.2 Premio per le soglie

Quando l'infedeltà scende sotto una soglia prevista, aggiungo dieci punti. Uso reached_milestones per assegnare il premio una sola volta per episodio. Con la soglia standard 0,01 c'è un solo traguardo, coincidente con il successo. Soglie più strette possono aggiungere altri traguardi.

quantum_rl/2.environment.py — estratto di step(); range e int sono funzioni Python, math.log10 è della libreria standard.

```
for exponent in range(2, int(-math.log10(self.threshold)) + 1, 2):
    milestone = 10 ** (-exponent)
    if distance < milestone and milestone not in self.reached_milestones:
        reward += self.bonus_milestone
        self.reached_milestones.add(milestone)
```

Con threshold=0,01 il ciclo controlla soltanto 10^{-2}: non assegna bonus a 0,1 o a 0,05. Il miglioramento fino alla soglia è già premiato dal logaritmo. I traguardi raggiunti vengono cancellati da `reset()` (quantum_rl/2.environment.py) all'inizio del tentativo successivo.

5.3 Bonus finale e numero di gate

Quando raggiungo il target aggiungo cento punti e cinque punti per ogni gate disponibile ma non usato. Se il limite è trenta e risolvo il problema con tre gate, questo secondo premio vale 5·27 = 135 punti.

quantum_rl/2.environment.py — estratto del ramo di successo di step(); len è una funzione Python.

```
gates_saved = self.max_gates - len(self.current_gates)
base_bonus = gates_saved * 5
reward += self.final_rew + base_bonus + exp_bonus
```

exp_bonus confronta la lunghezza della soluzione con il miglior risultato precedente dello stesso ambiente. Al primo successo vale zero. In seguito vale cinquanta punti se uso lo stesso numero di gate del record, aumenta se ne uso meno e diminuisce se ne uso di più.

```
exp_bonus = 50 · exp(−0,5 · max(L − L_best, −2))
```

L è la lunghezza attuale e L_best il record precedente. Il limite −2 evita una crescita eccessiva del bonus. Il record viene aggiornato dopo il premio e rimane tra gli episodi dello stesso ambiente. In valutazione creo invece un ambiente nuovo per ogni episodio.

Soluzione rispetto al record	exp_bonus
Primo successo nell'ambiente	0
Un gate in più	Circa 30,33
Stesso numero di gate	50
Un gate in meno	Circa 82,44
Almeno due gate in meno	Circa 135,91

Questi bonus arrivano soltanto al successo. Se finisco il budget senza raggiungere il target, non ricevo né i cento punti né i premi di brevità. Resta la somma delle reward ottenute durante le mosse.

5.4 Un esempio completo di calcolo

Considero un passaggio ipotetico da errore 0,020 a 0,005, con un circuito che risolve il target al terzo gate su un limite di sei. Suppongo che sia il primo successo di questo ambiente, quindi il bonus del record è zero. Non è un nuovo risultato sperimentale: serve solo a mostrare i conti.

Parte del punteggio	Calcolo	Punti
Miglioramento	$10 \ln(0{,}020 / 0{,}005)$	+13,86
Costo del gate	−1	−1
Soglia superata	0,005 < 0,01	+10
Successo	Bonus fisso	+100
Gate risparmiati	$5 \cdot (6-3)$	+15
Record	Primo successo	0
Reward della mossa	Somma dei contributi	Circa 137,86

Se, nello stesso ambiente, esistesse già un record di tre gate, si aggiungerebbero cinquanta punti. L'esempio mostra perché la reward finale non dipende soltanto dalla fidelity: contano anche lunghezza e successi precedenti. Il ritorno dell'episodio è invece la somma dei punteggi di tutte le mosse.

5.5 Perché considero anche il premio futuro

Riprendo Bell: prima di H(0) l'errore è 0,5; dopo H(0) è 0,75. La reward è $10 \ln(0{,}5/0{,}75) - 1 \approx -5{,}05$, senza bonus. Dopo CNOT(0,1), però, raggiungo Bell e ricevo il premio per il forte miglioramento e i bonus finali. L'agente deve imparare ad accettare un primo passo negativo quando permette di completare il circuito.

La reward favorisce questa ricerca, ma dipende anche dal budget e dai bonus. Per confrontare gli esperimenti guardo soprattutto fidelity, successi e gate. La baseline Qiskit usa una reward diversa, quindi i suoi punteggi non hanno la stessa scala di quelli del training.

6. Ottimizzazione degli angoli

Se l'agente sceglie una rotazione, devo trovare anche il suo angolo. Per farlo `step(action)` (quantum_rl/2.environment.py) richiama `_optimise_angles()` (quantum_rl/2.environment.py). Ottimizzo tutti gli angoli già presenti, non soltanto quello nuovo, mantenendo fissa la sequenza dei gate.

In `_optimise_angles()` (quantum_rl/2.environment.py) tratto gli angoli come manopole da regolare. Tengo fissa la ricetta, provo gli angoli attuali e misuro l'errore. Il gradiente indica in quale direzione conviene girare ciascuna manopola; Adam decide il piccolo aggiornamento da fare. Simulo di nuovo e ripeto. Solo alla fine scelgo il risultato migliore e calcolo la reward.

quantum_rl/2.environment.py — schema di _optimise_angles(); _state_torch() è nello stesso file, Adam e le operazioni sui tensori sono di PyTorch.

```
optimiser = torch.optim.Adam([parameters], lr=0.08)
# Schema del ciclo interno:
optimiser.zero_grad()
state = self._state_torch(parameters)
fidelity = torch.abs(torch.sum(torch.conj(target) * state)) ** 2
loss = 1.0 - fidelity.real
loss.backward()
optimiser.step()
```

Il limite standard è sessanta passi per tentativo. Conservo gli angoli migliori incontrati, perché l'ultimo aggiornamento non è sempre il migliore. Interrompo prima se l'errore è abbastanza basso, il gradiente è quasi nullo o compaiono valori non validi.

In `step()` (quantum_rl/2.environment.py) provo prima gli angoli correnti. Se non raggiungo il target, provo una seconda partenza casuale tra $-\pi$ e π. Confronto le fidelity e tengo il risultato migliore. Questa procedura, chiamata multi-start, aiuta a uscire da punti iniziali poco utili. Nel codice sono previsti al massimo due tentativi.

Ripeto l'ottimizzazione anche dopo un CNOT, se ci sono rotazioni nel circuito. Il CNOT non ha angoli, ma può rendere utili quelli dei gate precedenti. Se non ci sono rotazioni, salto Adam e simulo direttamente.

6.1 Il problema degli angoli a zero

Una nuova rotazione parte da zero. Su Bell e GHZ, finché mancano i CNOT necessari, una rotazione può non migliorare la fidelity. Adam può quindi lasciarla a zero o riportarla verso zero. L'agente continua ad aggiungere gate senza ottenere un premio utile.

Per GHZ servono collegamenti fra tutti e tre i qubit. Il problema è più difficile rispetto a Bell perché occorrono più scelte corrette prima del miglioramento. Ho affrontato questo blocco con partenze casuali, riottimizzazione dopo i CNOT e introduzione di H. Hadamard crea una sovrapposizione fissa, che Adam non può annullare cambiandone l'angolo.

7. Transformer e PPO

Ho usato un Transformer per leggere l'osservazione e proporre il prossimo gate. PPO è invece l'algoritmo che aggiorna i pesi della rete in base alle esperienze raccolte. I due componenti hanno quindi compiti diversi.

7.1 Come funziona la rete

La rete è nella classe MaskedTransformerModel, in quantum_rl/3.model.py. Il metodo `forward()` (quantum_rl/3.model.py) legge la scheda del tentativo. Ogni numero viene trasformato in un gruppo di 128 valori, chiamato token: è una rappresentazione che la rete può modificare e combinare con le altre. Aggiungo anche una posizione, per distinguere un'ampiezza dall'errore o da un gate passato.

I due blocchi di attention confrontano le informazioni della scheda. Per esempio, la rete può mettere in relazione la presenza di H(0), il qubit attivo e la possibilità di usare CNOT(0,1). Le otto teste di attention per blocco permettono di elaborare più relazioni. Queste relazioni vengono apprese con il training, non scritte come regole che impongono Bell.

quantum_rl/3.model.py — estratto di forward(); embedding, encoder, policy e value sono moduli PyTorch definiti nella stessa classe.

```
values = state_tensor.float().unsqueeze(-1)
tokens = self.token_embedding(values) + self.position_embedding
encoded = self.encoder(tokens).mean(dim=1)
logits = self.policy(encoded)
self._value_out = self.value(encoded).squeeze(1)
```

Riunisco le informazioni facendo la media dei token. Da questo riassunto ottengo due risposte. La policy dice quali mosse sembrano più promettenti. Il critic stima quanti punti potrei ancora ottenere: aiuta a capire se il tentativo è andato meglio o peggio del previsto. Prima della scelta assegno -10^9 alle mosse vietate dalla maschera, così non vengono selezionate.

Per esempio, dopo H(0) la rete può assegnare un punteggio alto a CNOT(0,1). RLlib converte i punteggi in una scelta. Il metodo `value_function()` (quantum_rl/3.model.py) restituisce invece la stima del critic: serve a PPO per valutare le esperienze, non per applicare un gate.

7.2 Come aggiorno la policy

PPO impara dai tentativi. Conservo la situazione di partenza, la mossa fatta e i punti ricevuti. Dopo molte mosse, confronto i risultati con le aspettative del critic. Se una scelta porta a risultati migliori del previsto, PPO tende a renderla più probabile; se va peggio, tende a ridurla. Gli aggiornamenti sono limitati per non cambiare strategia troppo bruscamente. Uso PPO di RLlib [1].

Nel caso di Bell, PPO vede anche il successo arrivato dopo la coppia H e CNOT. Può quindi imparare il valore del primo H pur avendo ricevuto una reward negativa per quella singola mossa. Gli angoli vengono adattati dentro l'ambiente; i pesi del Transformer vengono invece aggiornati da PPO usando gruppi di esperienze, nel training avviato da 2.train.py.

Parametro	Valore
Dimensione interna del Transformer	128
Blocchi / teste di attention	2 / 8
Learning rate PPO	0,0003
Batch di training	512 step
Minibatch	64 step
Epoche per aggiornamento	5
Coefficiente di entropia	0,02

L'entropia incoraggia l'esplorazione durante il training. In valutazione uso explore=False. Gli angoli possono comunque cambiare per le ripartenze casuali di Adam. Addestro la policy su un target alla volta: questi esperimenti non misurano l'uso della stessa rete su target nuovi.

8. Training e valutazione

8.1 I livelli dell'addestramento

Termine	Significato
Step	Scelgo e aggiungo un gate.
Episodio	Provo a costruire un circuito, dall'identità fino al successo o al limite di gate.
Iterazione PPO	Raccolgo dati da più episodi e aggiorno la rete.
Run	Eseguo un intero addestramento con una nuova rete iniziale.

2.train.py legge target, budget, iterazioni e worker dalla riga di comando. Registra ambiente e modello in RLlib e avvia Ray. A ogni iterazione salva nel CSV reward media, lunghezza media degli episodi e step raccolti.

Ogni venti iterazioni, e alla fine, chiamo `evaluate_best_circuit()` (2.train.py) per valutare la policy e `save_complete_checkpoint()` (2.train.py) per salvare il checkpoint. Il salvataggio passa da una cartella temporanea e verifica la presenza dei file principali. Salvo anche il miglior circuito osservato nelle valutazioni periodiche. La scelta privilegia la fidelity e, in caso di parità, il numero minore di gate.

8.2 Ripetizione degli esperimenti

3.run_repeated_training.py avvia prima il training e poi la valutazione. Ripete questa procedura per il numero di run richiesto. Ogni run ha una cartella run_i con checkpoint, cronologia e report JSON. Le run partono una dopo l'altra; il parallelismo riguarda i worker dentro ciascun training.

La procedura di una singola run è in `run_training_once()` (3.run_repeated_training.py): avvia 2.train.py, attende che finisca e lancia 5.evaluate.py sul checkpoint finale.

Nel riepilogo final_fidelity e gate_count descrivono il miglior episodio della run. total_reward è invece la reward media degli episodi valutati. success indica se il miglior episodio supera la soglia. Il tempo comprende training e valutazione finale.

8.3 Valutazione e grafici

5.evaluate.py ricarica soltanto la policy dal checkpoint e controlla che target e dimensioni siano compatibili. Esegue più episodi e salva fidelity media e migliore, reward media, gate medi e percentuale di successi.

Prima del caricamento uso `validate_checkpoint()` (5.evaluate.py) per controllare i file del checkpoint. Nel ciclo di valutazione richiamo poi `reset()` (quantum_rl/2.environment.py) e `step(action)` (quantum_rl/2.environment.py), cioè le stesse funzioni dell'ambiente descritte nella sezione 4.

Con --random posso usare lo stesso ambiente scegliendo a caso tra le azioni ammesse. Questa modalità mantiene rotazioni e ottimizzazione degli angoli: è utile per un confronto con l'agente a parità di strumenti. La baseline di 1.pre_trained_table.py usa invece Qiskit e i gate H, X, Y, Z e CX, senza angoli da

ottimizzare.

La baseline Qiskit sceglie gate casuali, senza apprendere. Dopo ogni gate controlla la fidelity F e si ferma al successo o al limite di gate. Il punteggio $10F - 0{,}1L$ premia la fidelity e penalizza il numero L di gate. I gate disponibili (X, Y, Z, H e CNOT) consentono di preparare Bell e GHZ, ma non un generico stato random: questo limita il confronto sui target random. La funzione run_random_circuit() (1.pre_trained_table.py) costruisce il circuito e restituisce esito, fidelity, numero di gate, tempo e sequenza.

4.plot_training_summary.py legge i CSV e crea due grafici: reward media rispetto alle iterazioni e fidelity finale rispetto alla baseline. Prima rimuove la riga Media, per non contarla come una run aggiuntiva.

Per leggere i dati uso `load_summary()` (4.plot_training_summary.py), che rimuove la riga finale di media, e `load_training_history()` (4.plot_training_summary.py), che controlla e ordina le iterazioni della singola run.

9. Esecuzione sul cluster HPC

Ho usato il cluster per eseguire più addestramenti, anche lunghi. SLURM assegna le risorse e avvia job_train.slurm. Il job attiva l'ambiente Conda quantum-rl e lancia lo script delle run ripetute.

```
# Risorse indicate in job_train.slurm
#SBATCH --nodes=1
#SBATCH --partition=cpu
#SBATCH --mem=16G
#SBATCH --time=1-00:00:00
#SBATCH --ntasks-per-node=6

# Comando Python del job
python 3.run_repeated_training.py --target custom   --target-file
targets/random_target_3_qubit_normalized.json   --qubits 3 --max-gates 30 --iterations 10   --
workers 2 --episodes 10 --runs 10   --checkpoint-root checkpoints_random3 --output
training_random3.csv
```

Ray usa due worker per raccogliere esperienze. Ogni worker simula i propri episodi; PPO usa poi i dati per aggiornare la rete. Nel codice imposto un thread PyTorch per worker, così evito che ogni processo provi a usare tutti i core del nodo.

Il costo maggiore nasce dalle simulazioni ripetute e dall'ottimizzazione degli angoli a ogni step. Con n qubit servono 2^n ampiezze: aumentando i qubit, memoria e lavoro crescono rapidamente. Più passi Adam possono migliorare gli angoli, ma rendono ogni decisione più lenta. Anche aumentare le iterazioni PPO richiede più tempo.

Per inviare un esperimento uso sbatch job_train.slurm; per controllare la coda uso squeue. Il job può continuare sul cluster anche dopo che ho spento il computer locale. Nei miei esperimenti uso CPU e un solo nodo.

10. Risultati

Riporto i risultati dei miei esperimenti. Ogni sperimentazione comprende dieci run. Presento prima le quattro baseline, poi i training con i rispettivi grafici.

PRIMA DEL TRAINING

Nella baseline casuale è disponibile anche il gate H. Senza H, usando soltanto X, Y, Z e CNOT, non potrei creare la sovrapposizione necessaria per Bell e GHZ: i successi su questi target sarebbero zero.

10.1 Stato target: Bell

Con i circuiti casuali ho ottenuto sette successi su dieci. La fidelity media è 0,7750 e il numero medio di gate è 18,1. Il riferimento breve è H(0) seguito da CNOT(0,1).

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Successo	1.00	13	8.70	CX(1, 0) -> X(1) -> H(1) -> ... (13 gate)
2	Successo	1.00	6	9.40	Y(0) -> Y(0) -> H(0) -> Z(0) -> CX(0, 1) -> Z(1)
3	Successo	1.00	10	9.00	X(1) -> H(0) -> Y(1) -> ... (10 gate)
4	Fallito (0)	0.25	30	-0.50	Z(0) -> Z(0) -> Y(1) -> ... (30 gate)
5	Successo	1.00	12	8.80	Y(1) -> Z(0) -> X(0) -> ... (12 gate)
6	Successo	1.00	24	7.60	Y(1) -> Y(1) -> Y(0) -> ... (24 gate)
7	Fallito (0)	0.25	30	-0.50	CX(1, 0) -> X(0) -> X(0) -> ... (30 gate)
8	Successo	1.00	6	9.40	Y(0) -> Y(0) -> H(0) -> Z(0) -> CX(0, 1) -> Z(1)
9	Successo	1.00	24	7.60	Y(1) -> Y(1) -> Y(0) -> ... (24 gate)
10	Fallito (0)	0.25	30	-0.50	Z(0) -> Z(0) -> Y(1) -> ... (30 gate)
Media	70.0%	0.7750	18.1	5.94	—

10.2 Stato target: GHZ

Per GHZ ho ottenuto cinque successi su dieci, con fidelity media 0,6000 e 24,5 gate medi. Un circuito di riferimento usa H e due CNOT.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Successo	1.00	13	8.70	CX(1, 0) -> X(1) -> H(1) -> ... (13 gate)
2	Successo	1.00	6	9.40	Y(0) -> Y(0) -> H(0) -> Z(0) -> ... (6 gate)
3	Successo	1.00	10	9.00	X(1) -> H(0) -> Y(1) -> ... (10 gate)
4	Fallito (0)	0.25	30	-0.50	Z(0) -> Z(0) -> Y(1) -> ... (30 gate)
5	Successo	1.00	12	8.80	Y(1) -> Z(0) -> X(0) -> ... (12 gate)
6	Successo	1.00	24	7.60	Y(1) -> Y(1) -> Y(0) -> ... (24 gate)
7	Fallito (0)	0.25	30	-0.50	CX(1, 0) -> X(0) -> X(0) -> ... (30 gate)

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
8	Fallito (0)	0.125	30	-1.75	H(0) -> Y(2) -> CX(0, 2) -> ... (30 gate)
9	Fallito (0)	0.25	30	-0.50	Z(1) -> X(0) -> Y(1) -> ... (30 gate)
10	Fallito (0)	0.125	30	-1.75	H(0) -> Y(2) -> CX(0, 2) -> ... (30 gate)
Media	50.0%	0.6000	24.5	3.85	—

10.3 Stato target: Random a 2 qubit

Non ho ottenuto successi: tutti i tentativi arrivano a trenta gate e la fidelity media è 0,2174. I gate della baseline non permettono di preparare un generico stato arbitrario.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Fallito (0)	0.4394	30	1.3939	Y(0) -> Y(1) -> CX(0, 1) -> ... (30 gate)
2	Fallito (0)	0.0076	30	-2.9242	CX(1, 0) -> CX(1, 0) -> Z(0) -> ... (30 gate)
3	Fallito (0)	0.2424	30	-0.5758	H(0) -> H(1) -> Z(0) -> ... (30 gate)
4	Fallito (0)	0.3030	30	0.0303	X(0) -> Z(0) -> Z(1) -> ... (30 gate)
5	Fallito (0)	0.3030	30	0.0303	X(0) -> CX(0, 1) -> Y(0) -> ... (30 gate)
6	Fallito (0)	0.1515	30	-1.4848	Y(0) -> H(0) -> Z(0) -> ... (30 gate)
7	Fallito (0)	0.0303	30	-2.6970	Z(0) -> Z(1) -> Y(0) -> ... (30 gate)
8	Fallito (0)	0.2424	30	-0.5758	H(0) -> H(1) -> Z(0) -> ... (30 gate)
9	Fallito (0)	0.1515	30	-1.4848	Y(0) -> H(0) -> Z(0) -> ... (30 gate)
10	Fallito (0)	0.3030	30	0.0303	X(0) -> CX(0, 1) -> Y(0) -> ... (30 gate)
Media	0.0%	0.2174	30.0	-0.8258	—

10.4 Stato target: Random a 3 qubit

Anche a tre qubit la baseline non raggiunge il target. La fidelity media è 0,0923 e tutti i tentativi usano trenta gate.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Fallito (0)	0.1335	30	-1.6651	Y(1) -> X(2) -> CX(1, 0) -> ... (30 gate)
2	Fallito (0)	0.0313	30	-2.6867	Z(1) -> CX(0, 2) -> Y(2) -> ... (30 gate)
3	Fallito (0)	0.1287	30	-1.7132	Y(1) -> CX(1, 0) -> X(1) -> ... (30 gate)
4	Fallito (0)	0.0590	30	-2.4095	H(2) -> X(1) -> Y(0) -> ... (30 gate)
5	Fallito (0)	0.0618	30	-2.3819	Z(0) -> CX(1, 2) -> Y(1) -> ... (30 gate)
6	Fallito (0)	0.0416	30	-2.5844	Z(2) -> Z(0) -> CX(2, 1) -> ... (30 gate)
7	Fallito (0)	0.1483	30	-1.5168	Y(0) -> CX(0, 2) -> Z(2) -> ... (30 gate)
8	Fallito (0)	0.1483	30	-1.5168	Y(0) -> CX(0, 2) -> Z(2) -> ... (30 gate)

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
9	Fallito (0)	0.1287	30	-1.7132	Y(1) -> CX(1, 0) -> X(1) -> ... (30 gate)
10	Fallito (0)	0.0416	30	-2.5844	Z(2) -> Z(0) -> CX(2, 1) -> ... (30 gate)
Media	0.0%	0.0923	30.0	-2.0772	—

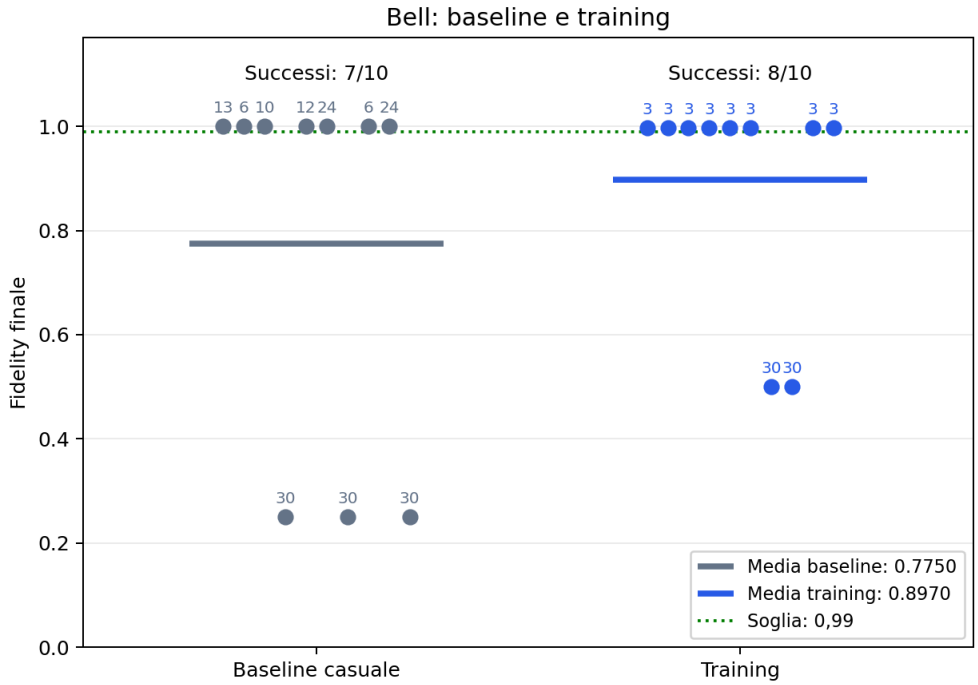
DOPO IL TRAINING

Nel primo training H non è disponibile: posso ottenere il target usando le rotazioni con angoli ottimizzati e i CNOT. Nel training GHZ aggiornato introduco anche H, per aiutare l'agente a superare il blocco descritto più avanti.

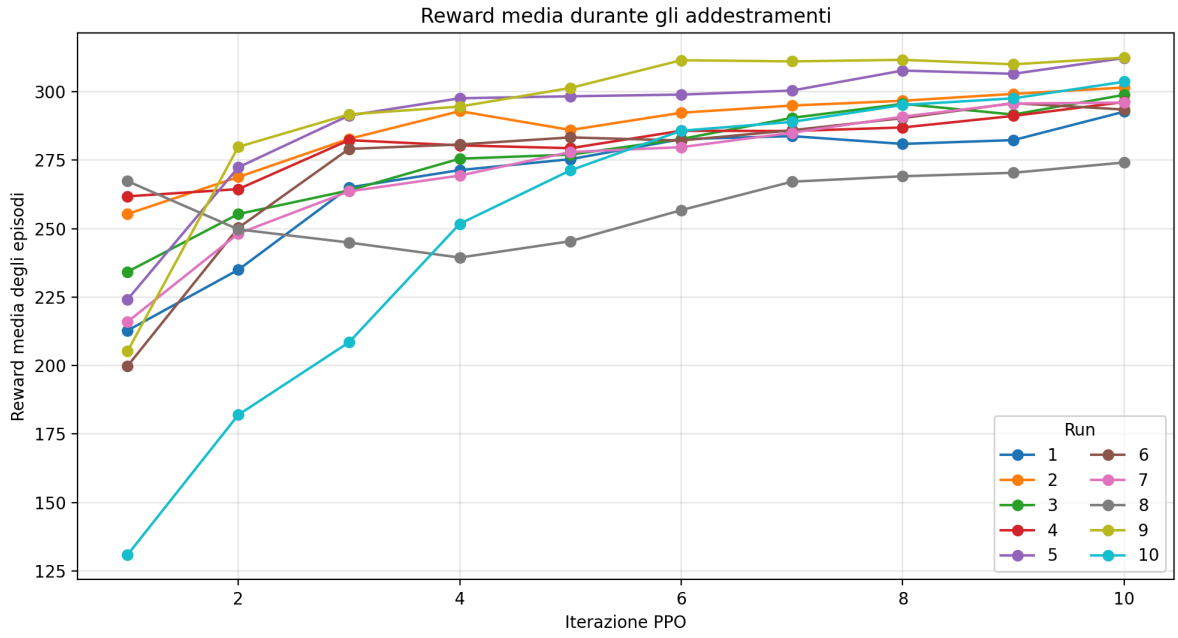
10.5 Bell: successo nell'80% delle run

Su Bell ho ottenuto otto successi su dieci, con fidelity media 0,8970. Le soluzioni riuscite usano tre gate; le due run fallite restano a 0,5. Il tempo totale è circa un'ora e trentadue minuti.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Successo	0.9963	3	291.00	CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)
2	Successo	0.9963	3	291.00	CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)
3	Successo	0.9963	3	291.00	CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)
4	Successo	0.9963	3	291.00	CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)
5	Successo	0.9963	3	291.00	CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)
6	Successo	0.9963	3	291.00	CNOT(0, 1) -> RY(1.692911, 0) -> CNOT(0, 1)
7	Fallito (0)	0.5000	30	-30.00	RY(0.000000, 1) -> RY(0.000000, 1) -> ... (30 gate)
8	Fallito (0)	0.5000	30	-30.00	RY(0.000000, 0) -> RY(0.000000, 0) -> ... (30 gate)
9	Successo	0.9963	3	291.00	CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)
10	Successo	0.9963	3	291.00	CNOT(1, 0) -> RY(1.692911, 1) -> CNOT(1, 0)
Media	80.0%	0.8970	—	226.80	—



Bell: confronto e andamento del training — Confronto della fidelity.



Run senza successo finale (migliore fidelity < 0,99): 7, 8

Bell: confronto e andamento del training — Reward durante il training.

10.6 GHZ: il primo training si blocca

Il primo training GHZ non riesce in nessuna run. La fidelity resta a 0,5 e i tentativi arrivano a trenta gate. Il tempo totale è circa un'ora e quarantasei minuti.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Fallito (0)	0.5000	30	-30.00	CNOT(0, 2) -> RY(0.000000, 0) -> ... (30 gate)
2	Fallito (0)	0.5000	30	-30.00	CNOT(2, 1) -> RY(0.000000, 2) -> ... (30 gate)

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
3	Fallito (0)	0.5000	30	-30.00	CNOT(1, 2) -> CNOT(0, 1) -> ... (30 gate)
4	Fallito (0)	0.5000	30	-30.00	CNOT(1, 0) -> CNOT(1, 2) -> ... (30 gate)
5	Fallito (0)	0.5000	30	-30.00	CNOT(1, 2) -> CNOT(0, 2) -> ... (30 gate)
6	Fallito (0)	0.5000	30	-30.00	CNOT(2, 1) -> CNOT(0, 2) -> ... (30 gate)
7	Fallito (0)	0.5000	30	-30.00	RY(0.000000, 2) -> RY(0.000000, 2) -> ... (30 gate)
8	Fallito (0)	0.5000	30	-30.00	CNOT(2, 0) -> RZ(0.000000, 1) -> ... (30 gate)
9	Fallito (0)	0.5000	30	-30.00	CNOT(0, 2) -> RY(0.000000, 0) -> ... (30 gate)
10	Fallito (0)	0.5000	30	-30.00	CNOT(1, 2) -> CNOT(1, 0) -> ... (30 gate)
Media	0.0%	0.5000	—	-30.00	—

Perché GHZ si blocca

Se le rotazioni restano a zero, non creo la sovrapposizione necessaria. I CNOT da soli non bastano a partire da $|000\rangle$. L'agente può così aggiungere gate senza cambiare il risultato, come spiegato nella sezione 6.

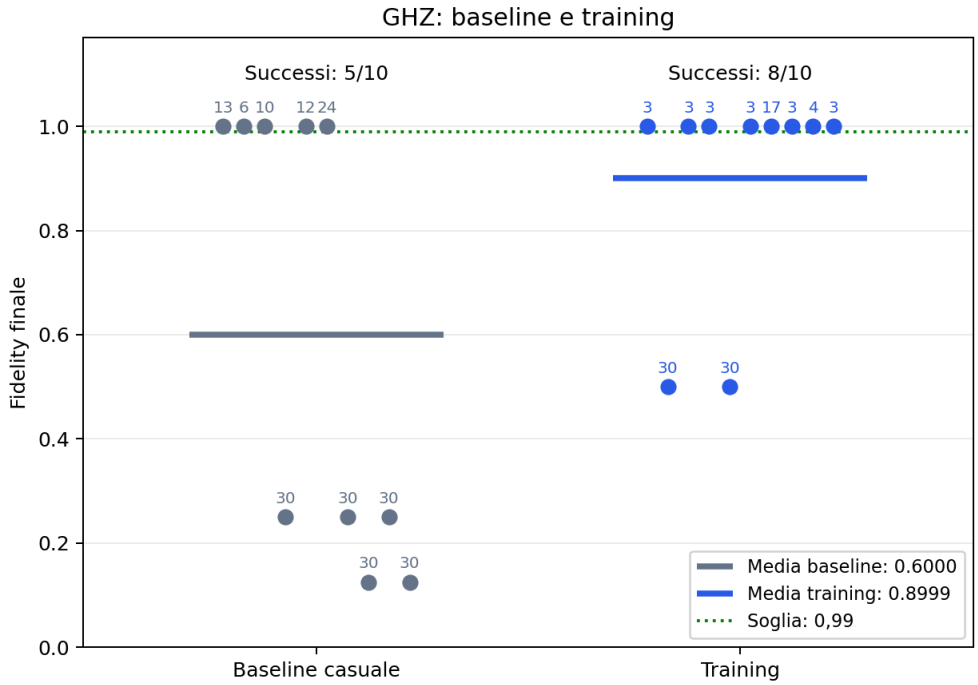
Come ho affrontato il blocco

Ho introdotto H per mantenere una sovrapposizione fissa, provato partenze diverse per gli angoli e riottimizzato le rotazioni dopo i CNOT. La maschera aiuta inoltre a evitare azioni inutili.

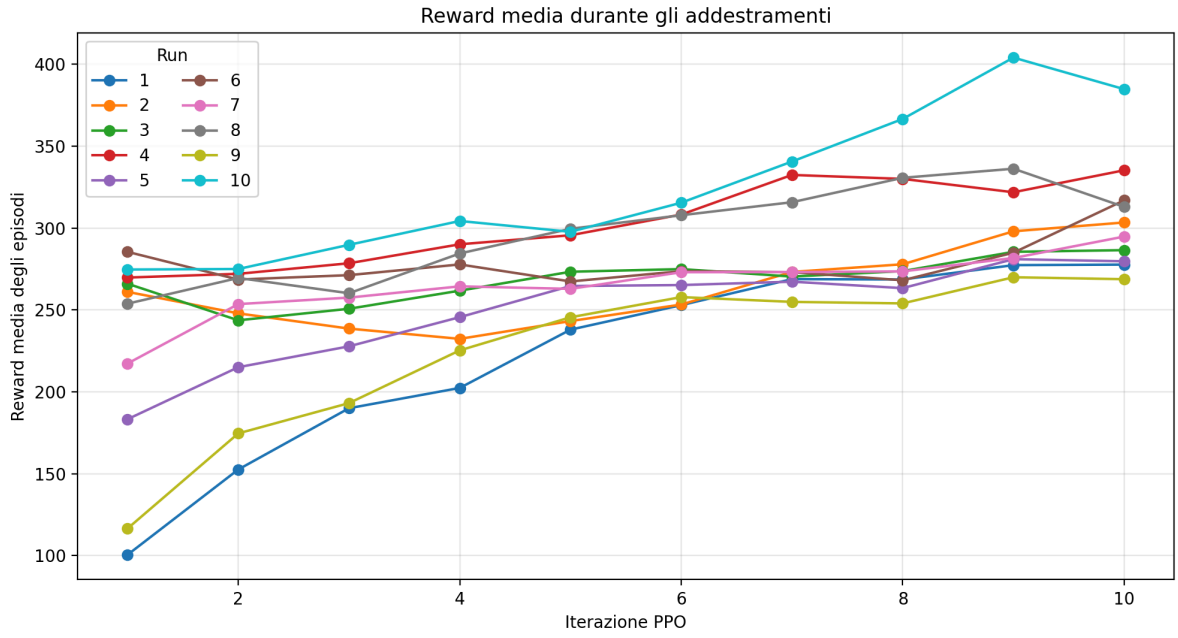
10.7 GHZ: da 0 a 8 run riuscite su 10

Dopo le modifiche ho ottenuto otto successi su dieci e fidelity media 0,8999. Diverse run trovano H(1) → CNOT(1,2) → CNOT(2,0), con fidelity 1. Il tempo totale è circa quattro ore e sedici minuti.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Successo	0.9995	3	311.73	RY(1.527506, 1) -> CNOT(1, 2) -> CNOT(2, 0)
2	Fallito (0)	0.5000	30	-30.00	RY(0.000000, 0) -> CNOT(0, 2) -> ... (30 gate)
3	Successo	1.0000	3	511.38	H(1) -> CNOT(1, 2) -> CNOT(2, 0)
4	Successo	1.0000	3	511.38	H(2) -> CNOT(2, 0) -> CNOT(0, 1)
5	Fallito (0)	0.5000	30	-30.00	RX(0.000000, 1) -> CNOT(1, 0) -> ... (30 gate)
6	Successo	0.9995	3	311.73	RY(1.527506, 2) -> CNOT(2, 1) -> CNOT(1, 0)
7	Successo	0.9995	17	227.73	RY(1.527506, 1) -> RZ(0.000000, 1) -> CNOT(1, 0)...
8	Successo	1.0000	3	511.38	H(1) -> CNOT(1, 2) -> CNOT(2, 0)
9	Successo	1.0000	4	505.38	RX(0.000000, 2) -> H(0) -> CNOT(0, 1) -> CNOT(0, 2)
10	Successo	1.0000	3	511.38	H(1) -> CNOT(1, 2) -> CNOT(2, 0)
Media	80.0%	0.8999	—	334.21	—



GHZ: confronto e andamento del training — Confronto della fidelity.



Run senza successo finale (migliore fidelity < 0,99): 2, 5

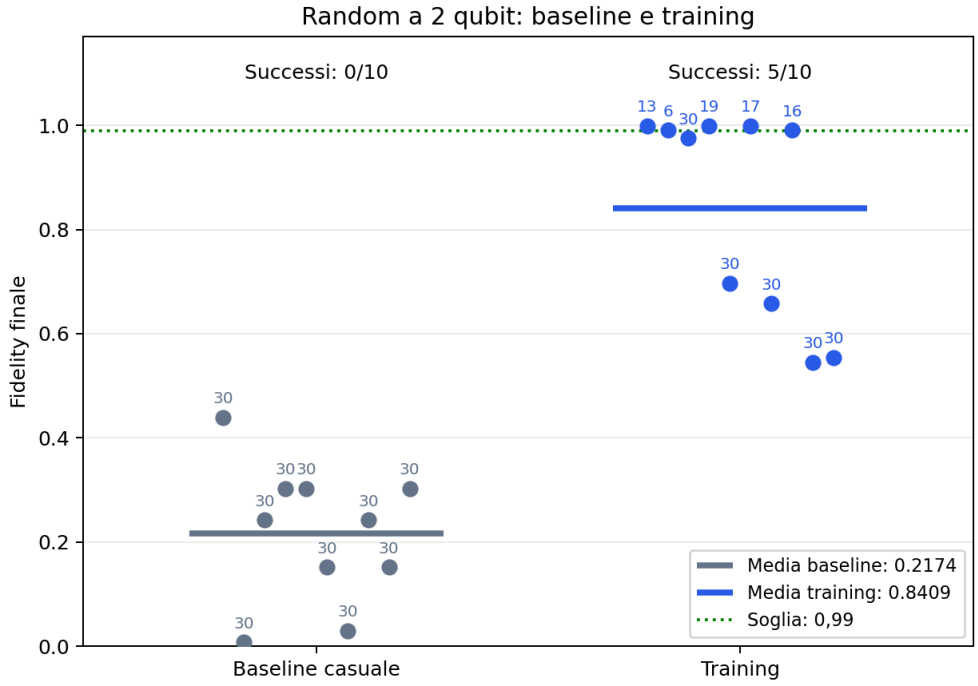
GHZ: confronto e andamento del training — Reward durante il training.

10.8 Random a 2 qubit: il 50% delle run riesce

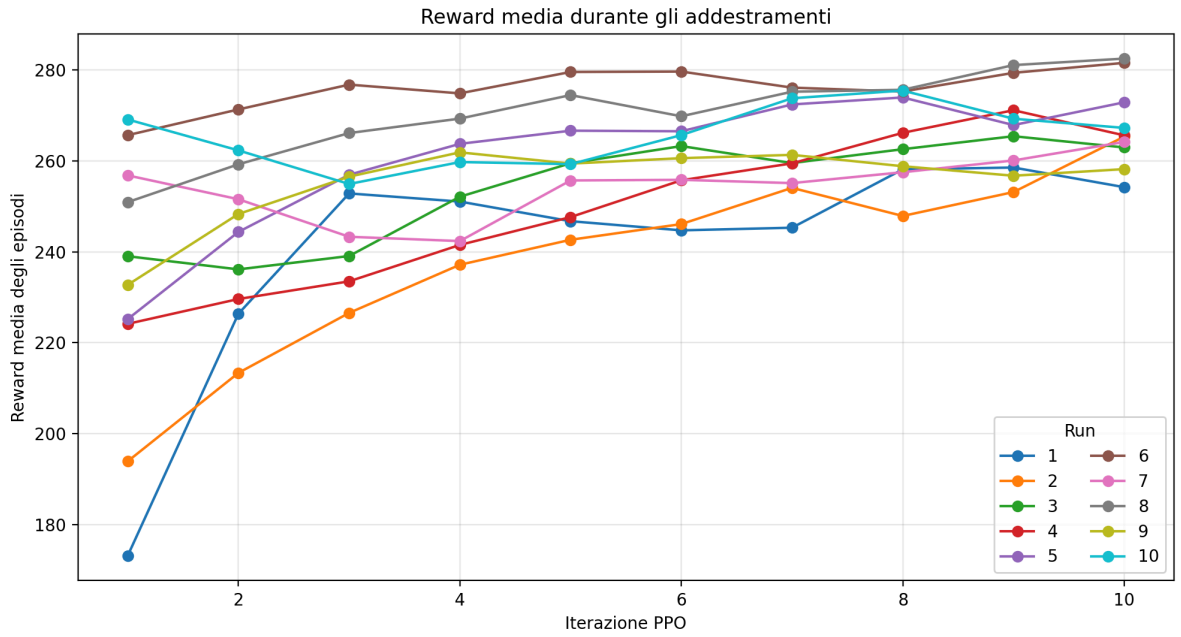
Ho ottenuto cinque successi su dieci e fidelity media 0,8409. La soluzione riuscita più corta usa sei gate con F≈0,9907. Il tempo totale è circa cinque ore e trenta minuti.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Successo	0.9991	13	3.78	RY(-2.753359, 1) -> CNOT(1, 0) -> RY(0.258590, 1) -> ...

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
2	Successo	0.9907	6	264.72	RY(1.349271, 0) -> CNOT(0, 1) -> RZ(0.327477, 1) -> RY(1.465698, 0) -> CNOT(0, 1) -> RZ(1.447523, 1)
3	Fallito (0)	0.9755	30	3.49	RY(1.474781, 0) -> CNOT(0, 1) -> RZ(1.383262, 0) -> ... (30 gate)
4	Successo	0.9990	19	208.22	RY(1.297238, 0) -> RY(2.208652, 1) -> CNOT(0, 1) -> ...
5	Fallito (0)	0.6970	30	-21.67	RX(-0.858371, 0) -> CNOT(0, 1) -> RX(2.685569, 0) -> ... (30 gate)
6	Successo	0.9993	17	228.72	RY(-1.221655, 1) -> CNOT(1, 0) -> RY(1.143484, 1) -> ...
7	Fallito (0)	0.6582	30	-22.88	RX(-1.862911, 0) -> RX(3.015778, 1) -> H(1) -> ... (30 gate)
8	Successo	0.9902	16	39.31	RY(-1.091360, 1) -> CNOT(1, 0) -> RY(0.218657, 1) -> ...
9	Fallito (0)	0.5455	30	-25.73	RY(0.314349, 0) -> RZ(-2.278639, 0) -> ... (30 gate)
10	Fallito (0)	0.5546	30	-25.52	RY(2.657912, 0) -> RZ(-1.323871, 0) -> ... (30 gate)
Media	50.0%	0.8409	—	65.25	—



Random a 2 qubit: confronto e andamento del training — Confronto della fidelity.

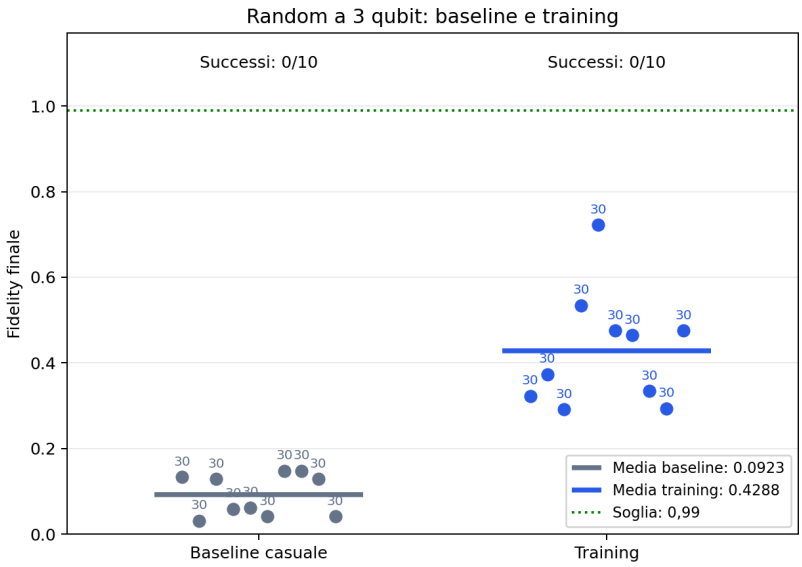


Random a 2 qubit: confronto e andamento del training — Reward durante il training.

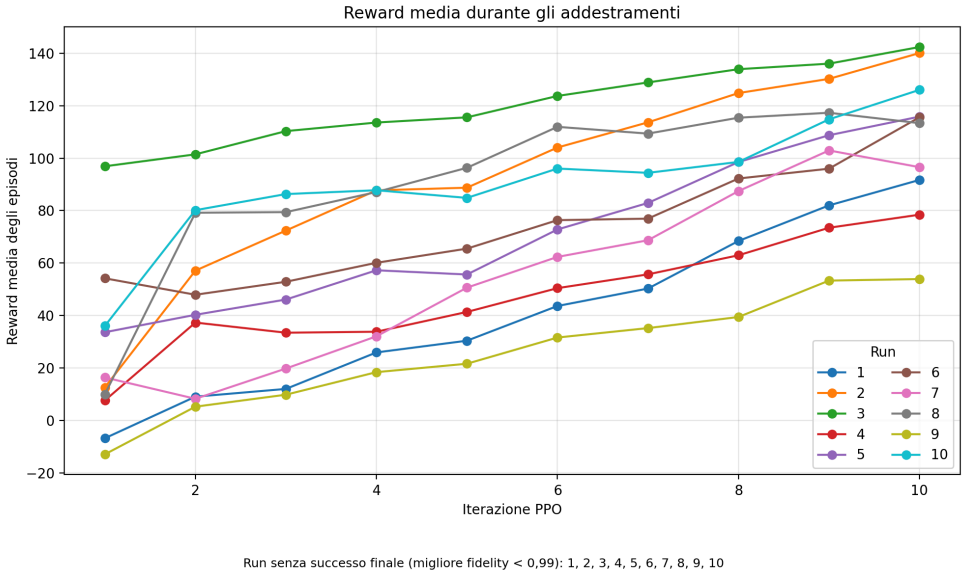
10.9 Random a 3 qubit: miglioramento senza successo

Non ho ottenuto successi, ma la fidelity media sale a 0,4288 e il miglior tentativo arriva a 0,7221. Tutti i circuiti usano trenta gate. Il tempo totale è circa dodici ore e trentanove minuti.

Run	Esito	Fidelity	Gate	Reward	Circuito trovato
1	Fallito (0)	0.3232	30	-27.64	RY(-1.452811, 2) -> CNOT(2, 0) -> RY(2.666358, 2)... (30 gate)
2	Fallito (0)	0.3737	30	-26.87	RY(-0.052656, 1) -> RX(2.052943, 0) -> RX(-1.882298, 2)... (30 gate)
3	Fallito (0)	0.2917	30	-28.10	RY(3.068356, 0) -> RX(1.578991, 0) -> RY(0.325931, 0)... (30 gate)
4	Fallito (0)	0.5341	30	-24.44	H(2) -> RY(2.805568, 2) -> H(2) -> RY(3.018255, 2)... (30 gate)
5	Fallito (0)	0.7221	30	-21.22	RY(-1.003208, 2) -> RX(-1.803071, 1) -> RY(-1.313247, 0)... (30 gate)
6	Fallito (0)	0.4752	30	-25.10	RX(0.237595, 1) -> RY(-1.732116, 2) -> RY(-1.739242, 0)... (30 gate)
7	Fallito (0)	0.4650	30	-25.29	RY(-1.343494, 2) -> RX(-1.818069, 0) -> RZ(-1.546965, 0)... (30 gate)
8	Fallito (0)	0.3341	30	-27.48	RY(-3.118705, 0) -> CNOT(0, 1) -> RY(-3.127674, 0)... (30 gate)
9	Fallito (0)	0.2940	30	-28.07	H(0) -> RY(-1.501816, 1) -> CNOT(1, 2) -> CNOT(2, 1)... (30 gate)
10	Fallito (0)	0.4751	30	-27.17	RY(2.366329, 0) -> RY(2.911689, 1) -> CNOT(0, 1)... (30 gate)
Media	0.0%	0.4288	—	-26.14	—



Random a 3 qubit: confronto e andamento del training — Confronto della fidelity.



Random a 3 qubit: confronto e andamento del training — Reward durante il training.

11. Conclusioni e possibili miglioramenti

Con questo progetto ho combinato la scelta dei gate tramite PPO con l'ottimizzazione degli angoli tramite Adam. La divisione dei compiti mi permette di cercare sia la struttura del circuito sia i parametri delle rotazioni. La reward guida l'agente verso alta fidelity e pochi gate.

Ho ottenuto buoni risultati su Bell e GHZ. Il target random a due qubit è riuscito in metà delle run; quello a tre qubit resta il caso più difficile. Il passaggio di GHZ da zero a otto successi mostra quanto contino la scelta dei gate e la gestione degli angoli.

Come sviluppo, aumenterei le iterazioni e confronterei una modifica alla volta: numero di tentativi Adam, presenza di H e maschera delle azioni. Userei anche la policy casuale nello stesso ambiente per misurare meglio il contributo di PPO. Più calcolo può aiutare, ma non garantisce risultati migliori.

Per rendere gli esperimenti più ripetibili, fisserei i seed e salverei insieme versioni delle librerie, comandi e metriche. Per studiare meglio la parte HPC, confronterei gli stessi esperimenti con diversi numeri di worker. Infine, potrei introdurre rumore e limiti sui collegamenti tra qubit per avvicinarmi ai dispositivi reali.

Riferimenti

[1] J. Schulman et al., *Proximal Policy Optimization Algorithms*, 2017. arxiv.org/abs/1707.06347.

[2] A. Vaswani et al., *Attention Is All You Need*, 2017. arxiv.org/abs/1706.03762. Riferimento per l'architettura Transformer.